Project Report GCA - CUFFT Convolutional Reverb

Stijn Vandendriessche
Master of Electronics and ICT Engineering Technology

Abstract: This report provides a detailed comparison of four unique methods used for calculating convolutional reverb, a critical technique used in digital audio processing. The convolution was carried out in both the time and frequency domain, using the CPU and the GPU. The results definitively show that, although Fast Fourier Transforms (FFT's) can considerably accelerate the process, the convolution of large audio data should not be performed on a CPU. Both GPU operations were significantly faster, with the FFT-based convolution slightly outperforming the time-domain convolution.
Keywords: CUDA, GPU, CPU, C++, CUFFT, Convolution, Reverb, FFT

1 Reverb

Reverb, short for reverberation, is an acoustic effect where sound waves reflect off surfaces, creating a series of echoes that blend with the original sound. In music production, reverb is used to enrich the listening experience by conveying a sense of physical space, whether it's the intimate echoes of a small room or the grand resonance of a large hall. In computer generated reverb, there are two main groups: algorithmic reverb and convolutional reverb.

1.1 Algorithmic Reverb

Algorithmic reverb is a digital effect used in audio production to simulate the reverberation of a space. It employs algorithms to create a series of echoes, trying to mimic real-life reverb.

1.2 Convolutional Reverb

Convolutional reverb, in contrast, relies on impulse responses to generate its effect. This method captures the unique characteristics of a specific location, providing a high degree of realism in the resulting sound. Convolutional reverb is often praised for its accuracy and ability to replicate the natural acoustics of venues, ranging from small rooms to large cathedrals. However, it is very computationally intensive. In this project, we will focus on convolutional reverberation.

2 Convolution

Convolution, and more specifically, discrete convolution, is a mathematical operation defined by the following formula:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k] \tag{1}$$

This operation can be computationally intensive. Often, a Discrete Fourier Transform (DFT) or Fast Fourier Transform (FFT) is employed since convolution in the time domain corresponds to multiplication in the frequency domain. We will be using both discrete convolution in the time domain and multiplication in the frequency domain in this project.

3 Convolutional Reverb as a LTI System Response

When we consider a specific location or space as a system, we can determine its acoustic output $y(t)$ by convolving a given input signal $x(t)$, which could be any sound generated, with the impulse response $h(t)$ of this location (see Figure 1).

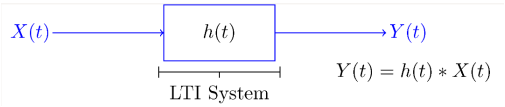


Figure 1: *Visual representation of an LTI system*

When we execute a frequency sweep at the chosen location and record the response, we can easily calculate the impulse response using the following formulas:

$$y(t) = x(t) * h(t) \tag{2}$$

$$\mathcal{F}\{y(t)\} = \mathcal{F}\{x(t) * h(t)\} = X(\omega) \cdot H(\omega) \tag{3}$$

$$H(\omega) = \frac{Y(\omega)}{X(\omega)} \tag{4}$$

$$h(t) = \mathcal{L}^{-1}\left\{\frac{Y(\omega)}{X(\omega)}\right\} \tag{5}$$

Now, if we convolve any dry audio signal $x(t)$ with $h(t)$, we obtain $y(t)$. In simpler terms, this means we have calculated how any particular sound would sound like in the recorded location.

4 The Project

To demonstrate convolutional reverb, we used dry audio of a male speaker and the impulse response of a church. The result of all four subprojects is the same: the male voice, initially sounding dry, is transformed to sound as if it were recorded in the church. The .wav files for these demonstrations can be found in each subfolder of the CUDA project.
Both the impulse response as the audio data are sampled at 8000 samples per second, with 16 bits per sample using 1 channel. Note that it is advisable to keep these parameters the same for both signals.

4.1 The Four Strategies

Convolutional reverb was implemented using four different techniques:

CPU-based Naive Convolution This technique is a straightforward implementation of formula 1. It is easy to understand since it is executed in the time domain and operates serially.

CPU-based FFT Convolution This technique utilizes the FFTW library in C++ to perform the Fast Fourier Transform and its inverse. This is combined with a CPU function that performs element-wise multiplication.

GPU-based Naive Convolution Similar to CPU-based naive convolution, this technique performs convolution in the time domain. It implements formula 1 but in a GPU kernel, allowing for parallelization. Note that this technique could be improved by storing the impulse response in constant memory and tiling over the audio data in shared memory to limit slow global memory operations.

GPU-based FFT Convolution This technique leverages the CUFFT library developed by NVIDIA to execute the Fast Fourier Transform and its inverse. CUFFT is an extension of the CUDA framework, so the FFT and IFFT are highly parallelized on the GPU. This is combined with an element-wise multiplication kernel, also on the GPU.

5 Results

The four approaches were tested by running the convolution 100 times and measuring the execution time. The results are shown in Figure 2.

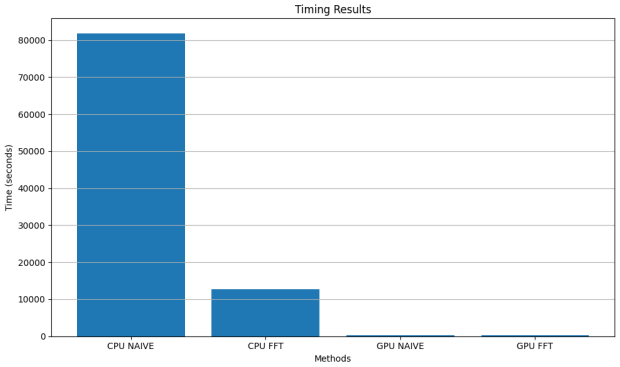


Figure 2: *Execution time of the four convolution approaches*

As observed in Figure 2, both CPU approaches exhibit relatively slow performance. The naive approach took 22 minutes, 42 seconds, and 49 hundredths, whereas the FFT approach was significantly faster, completing in only 3 minutes, 33 seconds, and 1 hundredth.

However, the GPU approaches demonstrated superior performance. The naive GPU technique completed in 5 seconds and 45 hundredths, while the FFT GPU approach performed even better, finishing in just 4 seconds and 40 hundredths.

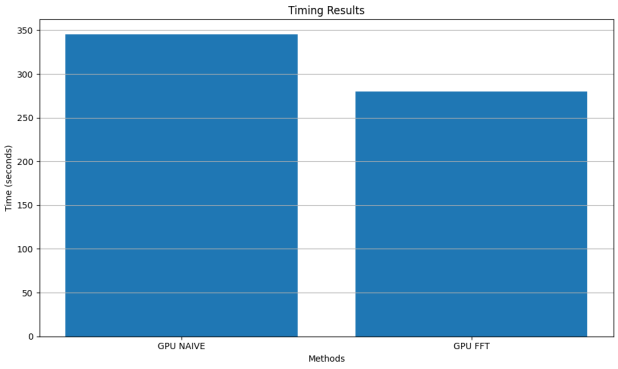


Figure 3: *Execution time of the two GPU approaches*

6 Conclusion

This report has thoroughly explored and compared four distinct methods for calculating convolutional reverb, a crucial technique in digital audio processing. Through the analysis, it has become evident that convolutional reverb, while computationally intensive, can be efficiently executed using GPU-based techniques. This research contributes valuable insights into the practical implementation of convolutional reverb techniques and highlights the significance of GPU acceleration in audio processing applications.

7 Github link

`https://github.com/StijnVandendriessche1/CudaConvolutionalReverb.git`